

Framework Selection Report

Pangasinan Heritage Digital Showcase

Course: Elective 4 - Special Topics in IT Student: [STUDENT NAME - replace before submission] Date: 23 August 2026

Selected framework: React with Next.js 14 App Router Alternative evaluated: Vue with Nuxt 3

Executive Summary

This report evaluates React with Next.js 14 App Router and Vue with Nuxt 3 for the Pangasinan Heritage Digital Showcase. Both candidates can produce a component-based, statically generated tourism site. Both can support responsive layouts, accessible client-side interaction, and deployment to a content delivery network.

The decision is therefore based on the needs of this project rather than a claim that one framework is universally superior. The project requires two polished pages, an explicit Atomic Design hierarchy, selective cinematic interaction, client-side search, static export, and GitHub Pages deployment. A weighted decision matrix gives Next.js 14 a project score of 93.0/100 and Nuxt 3 a project score of 84.5/100. Next.js is selected because its App Router maps cleanly to the required page structure, its Server Component default limits client JavaScript, its static export is compatible with the required hosting model, and React aligns directly with the activity specification.

No cross-framework bundle benchmark was fabricated. The only implementation measurement reported here is the local Next.js production build output from this project. Framework scores are transparent project judgments on a five-point scale, not laboratory measurements.

1. Project Context and Decision Method

The showcase promotes Pangasinan's landscapes and heritage sites through an editorial home page and a searchable collection. It must remain usable on mobile data, understandable for an oral academic defense, and deployable as static files under a GitHub repository subpath.

Seven criteria were weighted according to their effect on project success. Performance/static delivery and project suitability receive the largest weights because Activity 2 requires static generation and GitHub Pages, while the visual brief still expects a smooth media-led experience. Developer velocity and component architecture follow because the work includes both implementation and a documented Atomic Design library.

Score definitions

5.0	Excellent fit with direct, well-documented support and little project friction
4.0	Strong fit with manageable tradeoffs or some additional configuration
3.0	Adequate fit; meaningful compromises or extra work are expected
2.0	Weak fit; major workarounds would be needed
1.0	Does not reasonably satisfy the project need

Each weighted contribution is calculated as (score / 5) x criterion weight.

2. Quantitative Decision Matrix

Performance and static delivery	20%	4.5	18.0	4.5	18.0
Project suitability	20%	5.0	20.0	4.0	16.0
Developer velocity	15%	4.5	13.5	4.0	12.0
Component architecture	15%	4.5	13.5	4.5	13.5
Ecosystem maturity	10%	5.0	10.0	4.5	9.0
Documentation and community	10%	5.0	10.0	4.5	9.0
Learning curve	10%	4.0	8.0	3.5	7.0
Total	100%		93.0		84.5

Score rationales

Performance and static delivery - 4.5 each. Next.js supports static export through output: "export"; Nuxt supports pre-rendered/static output through its rendering and generation tools. Either can ship static assets without a runtime server. Both still require disciplined image sizing, client-boundary control, and dependency restraint, so neither receives an automatic perfect score.

Project suitability - Next.js 5.0; Nuxt 4.0. The activity explicitly requires React through Next.js 14 App Router, making Next.js the direct compliance path. Nuxt is technically capable, but selecting it would conflict with the stated implementation requirement even though it is a valid comparison candidate.

Developer velocity - Next.js 4.5; Nuxt 4.0. Both provide file-based routing, development servers, production builds, and mature tooling. Next.js scores slightly higher because the required team output, examples, and assessment language are already centered on React/Next.js, avoiding a translation step during implementation and defense.

Component architecture - 4.5 each. React composition and Vue Single-File Components both support Atomic Design. Next.js Server and Client Component boundaries offer an additional way to limit browser JavaScript; Nuxt provides strong component conventions and auto-import capabilities. The final architecture quality depends more on folder discipline and prop/data design than on framework branding.

Ecosystem maturity - Next.js 5.0; Nuxt 4.5. React and Next.js provide a very broad ecosystem and long-standing integration support. Vue and Nuxt are also mature and production proven, but the React ecosystem provides a small advantage for this project's likely future extensions and institutional familiarity.

Documentation and community - Next.js 5.0; Nuxt 4.5. Both projects maintain substantial official documentation. Next.js receives the small advantage because official App Router, static export, image, and deployment guidance directly maps to this implementation.

Learning curve - Next.js 4.0; Nuxt 3.5. Both require understanding routing, rendering modes, reactivity, and build configuration. React familiarity reduces local learning cost for the selected implementation. Next.js App Router's Server/Client distinction is a real concept to learn, so it does not receive 5.0. Nuxt's conventions are approachable, but Vue reactivity and Nuxt-specific conventions would add a second learning transition for a React-targeted activity.

3. Bundle Size and Performance

Framework package size alone does not determine the visitor experience. Route JavaScript, hydration boundaries, imagery, fonts, CSS, caching, and third-party scripts all matter. For this project, scenic media is the largest likely payload, so local WebP files, explicit display sizes, lazy loading, and the absence of animation libraries have more direct value than quoting an unverified framework download size.

Next.js App Router renders page structure and content statically while only interactive islands use "use client". The implemented client boundaries are the mobile header, cinematic controller, carousel, search/grid controller, and search input. Cards, data, footer, typography, and page layouts remain static. The final output is exported as HTML, CSS, JavaScript, and

images for GitHub Pages.

The verified local production build reported approximately 106 kB first-load JavaScript for the home route and 104 kB for `/heritage` during the first completed build. These are Next.js build-report values from this repository, not compressed network transfer measurements and not a comparison against Nuxt. No equivalent Nuxt implementation was built, so no exact bundle claim is made for it.

Nuxt can also pre-render routes and reduce client work with careful rendering choices. It would be a performance-capable alternative. The practical conclusion is parity at the framework-capability level, with Next.js favored because its required implementation is already structured around small client islands.

4. Developer Velocity

Next.js supplies file-based App Router pages, route metadata, TypeScript support, CSS Modules, static export, and framework-aware links and images. These reduce integration work without imposing a visual component library. The project can therefore keep its tourism identity instead of adapting a generic template.

Nuxt similarly offers file-based routing, strong defaults, composables, and automatic component conventions. It can be very productive for a Vue-experienced team. For this assignment, however, using Nuxt would require translating the required React component set and explaining why the implementation diverges from the specified framework. That reduces effective velocity even if authoring Vue components is concise.

5. Ecosystem Maturity

React's ecosystem covers accessibility testing, browser automation, static hosting, content systems, and future API integration. Next.js has established patterns for static and server-rendered applications. The project deliberately uses very little of that ecosystem at runtime: React, Next.js, browser APIs, and CSS are sufficient.

Vue and Nuxt also have mature communities, modules, and deployment integrations. Their ecosystem is not a project blocker. Next.js receives the higher score because of direct framework compliance and the breadth of React-aligned learning material likely to support maintenance by future students.

6. Learning Curve

The selected architecture avoids advanced abstractions. Tourism records are plain typed objects. Search uses `useState` and `useMemo`. Motion uses a single effect with `requestAnimationFrame`. Components accept straightforward props. This makes the project defensible in an oral presentation.

The main Next.js concepts a student must explain are App Router folders, static Server Components, targeted Client Components, `output: "export"`, and repository-aware asset paths. Nuxt would replace these with Vue templates, reactivity, composables, Nitro/Nuxt rendering concepts, and its generation conventions. Neither is trivial; existing React alignment makes Next.js the lower-risk teaching path here.

7. Component Architecture

Atomic Design is framework independent. The implementation uses:

- atoms for Button, Typography, Icon, responsive Image, and color tokens;
- molecules for Heritage Card, Search Form, and Navigation Item;
- organisms for Heritage Grid and Header Navigation;
- sections for the cinematic hero, destination carousel, and footer.

React props keep the content/presentation boundary explicit, while TypeScript verifies the `HeritageSite` model. The App Router allows pages to remain static and send props into narrow client components only when interaction is required. Nuxt could create a comparable hierarchy with Vue components, but it offers no decisive architectural advantage for this exact

brief.

8. Documentation and Community Support

Official Next.js documentation describes the App Router, static exports, image behavior, and GitHub Pages-oriented deployment concepts. Official React documentation covers component composition and state. Official Nuxt documentation clearly explains rendering modes and its component directory. Both candidates therefore meet the documentation threshold.

The project also retains `ARCHITECTURE.md`, a complete `PROJECT-CHECKLIST.md`, code-level naming, an Atomic Design manual, image-license records, and reproducible test commands. Framework documentation is only one layer; project-specific documentation is what makes the final submission maintainable.

9. Suitability for the Pangasinan Showcase

Next.js 14 is suitable because it satisfies the required React implementation, supports the two App Router routes directly, statically generates the content, limits JavaScript through server-first composition, and exports a GitHub Pages-ready artifact. It also supports independent metadata for Discover and Heritage pages and a clean path to future API-backed content without requiring a backend today.

Nuxt 3 would remain a credible alternative for a Vue-oriented brief. It is not rejected for poor quality or poor performance. It is not selected because the assignment's direct React/Next.js requirement, the planned component library, and the student's expected explanation all make Next.js the more appropriate project decision.

10. Risks and Mitigations

Cinematic motion increases CPU or motion discomfort	Transform/opacity-only motion, limited pointer work, <code>requestAnimationFrame</code> , and a complete reduced-motion layout
Scenic media dominates transfer size	Local resized WebP images, aspect ratios, below-fold lazy loading, no 4K assets
GitHub Pages repository subpath breaks assets	Build-time <code>basePath</code> , <code>assetPrefix</code> , shared public-path helper, and Actions build under repository context
Excessive client JavaScript	Static pages and content; "use client" only around actual interaction
Framework version advisories	Static-only hosting removes the Next server attack surface; <code>SECURITY-NOTES.md</code> records the need to upgrade before server deployment
Content becomes stale or overclaims facts	Short factual-safe copy, source separation, and explicit visitor-information caveat

11. Final Recommendation

Select React with Next.js 14 App Router. The 93.0 weighted score reflects direct academic compliance and strong technical fit, not universal superiority. The implementation demonstrates that the choice can produce a cinematic yet accessible and lightweight static tourism experience, while preserving a simple Atomic Design structure that a student can explain and maintain.

References

- Next.js. *App Router Documentation (Version 14).* <https://nextjs.org/docs/14/app>
- Next.js. *Static Exports (Version 14).* <https://nextjs.org/docs/14/app/building-your-application/deploying/static-exports>
- React. *Describing the UI.* <https://react.dev/learn/describing-the-ui>
- Nuxt. *Rendering Modes.* <https://nuxt.com/docs/3.x/guide/concepts/rendering>
- Nuxt. *Components Directory.* <https://nuxt.com/docs/3.x/guide/directory-structure/components>
- GitHub Docs. *Using custom workflows with GitHub Pages.* <https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages>